

Your questions from today's session

You asked 88 questions in three hours, which is genuinely the best sign a session can have. I have grouped them — a lot of you asked the same thing — and answered every one below. Your name is on each question you asked, so search this document for your own name to find your answers.

READ THIS FIRST

Two documents do most of the work here, and several answers just point at them:

- **Build the whole thing, end to end** — every command, every file in full, eight checkpoints, and a troubleshooting table. If you missed steps today, work through this instead of the recording. It is complete and standalone.
- **No stupid questions** (the learner companion) — the concepts explained properly: what a container is, what activation does, why CORS exists.

Session repo: github.com/amitpandeygit/llm-workshop

MY MISTAKE — PLEASE READ

The `requirements.txt` I gave you pins `psycpg[binary]==3.2.3`, and that version has no installable build for Python 3.14. If your `pip install` failed today, it was my pin, not your machine. The corrected file is in the next section, and the guide has been updated. Thank you to those who debugged it live and posted working versions — that was genuinely useful.

SECTION 1

The install error, and Python versions

pip install fails with “Could not find a version that satisfies the requirement psycopg-binary==3.2.3”

SANDEEP MADDU · PRATEEK SINGHAL · KRISHNA SAGAR · ANNUREKHA SB · ANONYMOUS

This is my error, not yours. I pinned `psycopg[binary]==3.2.3`, which was released before Python 3.14 existed, so there is no matching build for it — pip lists the versions that *do* exist (3.2.10 and up) and stops. Replace `api/requirements.txt` with this:

```
fastapi>=0.115.0      # the web framework
uvicorn[standard]>=0.30.6 # the server that listens on a port
pydantic>=2.9.2        # request/response validation
psycopg[binary]>=3.2.10 # PostgreSQL driver
pytest>=8.3.3          # test runner
httpx>=0.27.2          # required by FastAPI's TestClient
```

Then `pip install -r api/requirements.txt` again. Prateek and Krishna both posted working sets in the chat — Krishna's exact pins for 3.14 and Prateek's for 3.13 — and both are correct. Annurekha's write-up of downgrading to 3.13 also works, and is the more conservative choice if you would rather not touch versions.

Worth understanding the trade-off, because it matters beyond today: `==` pins give you exact reproducibility, which is what you want in a real project. The catch is that an exact pin is only installable where a build exists for your Python. `>=` is more forgiving and right for a teaching repo where everyone is on a different Python.

I have Python 3.14 / 3.13 — is that a problem? Is 3.11 the stable version?

AMIT SARKAR · KRISHNA SAGAR · LAVANYA KOONALA

Anything 3.11 or later is fine, and newer is not a problem — **3.11+** was a minimum, not a target. Amit, your set (Python 3.14.5, Docker Desktop 5.3.1, Git 2.50.1) is exactly what you need; nothing missing.

The one caveat is the one above: the very newest Python sometimes runs ahead of the libraries, because each package has to publish a build for each Python version. Krishna, you are right that it works on 3.14 — with the relaxed pins. If you want the quietest possible ride for this course, 3.12 or 3.13 is the sweet spot.

I have Anaconda / conda. Is that okay? You have a "base" env and I don't.

SHUBH MANDALIA · MADHAV BHETANABHOTLA

Anaconda is fine to have installed, and yes — that (base) in my prompt is conda, which is why yours looks different if you installed plain Python. Nothing is wrong with your setup.

For this course please still use `python -m venv` as the guide shows, rather than a conda environment. Two reasons: `venv` is what ships with Python so everyone has it, and our Dockerfile installs from `requirements.txt`, which is pip's format. Mixing conda and pip in one environment is a well-known source of confusing breakage. If conda's base environment activates automatically and you find it distracting: `conda config --set auto_activate_base false`.

SECTION 2

Code, links and copying from the PDF

Please share the repo link / the code / the commands

VIKRANT · MOULI CHOUDHURY · KAUSTAV KARMAKAR · SUBHRAJIT PAUL · JISHAN BAIG

Repo: github.com/amitpandeygit/llm-workshop. Every command, in order, with expected output, is in **Build the whole thing, end to end** — that is the document to work from. Subhrajit, to answer directly: that one, not the slides.

I can't copy and paste code out of the PDF — some of it looks like an image

MAHENDRA · NARSI VELDANDA

Fair, and my fault for not saying so up front: **clone the repo instead of copying from the PDF**. Every file in the guide exists there as a real file. The PDF is for reading and understanding; the repo is for running.

Both of you asked for specific files, so here they are. `requirements.txt` is in Section 1 above. `.gitignore`, at the project root:

```
# Python
.venv/
__pycache__/
*.pyc
.pytest_cache/

# Node
node_modules/
.next/

# Editor / OS
.vscode/
.DS_Store
```

Please share the corrected main.py

JISHAN BAIG

It is **Part 4** of the build guide — the file labelled “version 2 — PostgreSQL, complete file, replaces version 1”. That is the corrected one, in full, with comments. The version 1 earlier in the guide is the in-memory one we started with; it is meant to be replaced.

“Permission denied” when cloning the repo

AJAY SARSWAT

You sorted this yourself during the session, so this is here for anyone hitting it later: that error means Git tried to use SSH without a key configured. Clone over HTTPS instead — `git clone https://github.com/...` rather than `git@github.com:...`

Which IDE are we using? And is that terminal just cmd on Windows?

MOULI CHOUDHURY · ANONYMOUS

VS Code, with the official **Python** extension — that extension is what gives you the type-hint autocomplete and the inline red squiggles I kept pointing at. Install it if you have not.

The terminal is VS Code's own integrated terminal (Ctrl+`), which on Windows is usually PowerShell. Use that rather than a separate window — it opens in your project folder, which removes a whole class of “wrong directory” errors. Run *all* commands there, including the Docker ones; you never need Docker Desktop's terminal.

SECTION 3

Docker: what it is, and installing it

Can you give a simple example of what Docker actually does?

KRISHNA SAGAR

Today's database is the example. Nobody in the room installed PostgreSQL — yet everyone ran the same PostgreSQL 16, with the same schema and the same two seed rows, on Windows, macOS and Linux alike. One line in a file, one command, identical result. That is the whole value: the software travels with the project instead of being installed on the machine.

The mental model: an **image** is a frozen recipe (`postgres:16-alpine`); a **container** is a running instance of it. One image, many containers — like a class and its objects.

Is a Docker container basically just a virtual environment?

ANUSHKA SEHRAWAT

Same instinct, different scope — and worth being precise about, because the distinction comes up constantly.

- A **virtual environment** isolates Python packages. Nothing else. It uses your machine's Python, your machine's OS, your machine's libraries.
- A **container** isolates the whole userland — the OS libraries, the Python installation, the packages, the filesystem. It ships its own Linux.

So a venv means “my Python packages don't collide with yours”, while a container means “this runs identically anywhere”. Today we used both: a venv while developing locally, a container to ship it. And a container is still not a virtual machine — it shares your machine's kernel rather than simulating a computer, which is why it starts in a second.

Containers sit on my local disk — so they still consume space, right?

ANUSHKA SEHRAWAT

Yes, and more than people expect — images are the bulk of it, and old ones do not clean themselves up. Check and reclaim:

```
docker system df          # what is using space
docker system prune       # remove stopped containers, unused networks
docker system prune -a    # ...and all unused images (more aggressive)
```

Careful with `-a`: it will delete images you would otherwise have to download again. Never add `--volumes` unless you intend to destroy stored database data.

docker.com is blocked by my organisation

AKSHATA DV

That is a corporate network policy and nothing you can fix from your side. Options, roughly in order of ease: install from a personal machine or a phone hotspot; ask IT for an exception, since Docker Desktop is standard developer tooling; or use one of the alternatives in the next answer, which sometimes are not on the same blocklist. Message me if none of these work and we will find you a path.

Can I use Rancher Desktop or Podman instead of Docker Desktop?

AMAN ANAND

Yes, both work, and Rancher Desktop in particular is a close drop-in — it provides the same `docker` and `docker compose` commands, so everything in the guide applies unchanged.

Podman also works but you may need `podman compose` (or the `podman-compose` package), and rootless mode occasionally needs extra care with volume mounts.

Both are legitimate choices, often for licensing reasons in larger companies. My only ask: the guide is written and tested against Docker Desktop, so if something behaves oddly, check it against Docker Desktop before assuming the guide is wrong.

“Virtualization Support not detected”, and Docker login problems

ANONYMOUS · AYAN BHATTACHARYA

The virtualization error is a BIOS/UEFI setting, not a Docker problem. Reboot into your firmware settings and enable **Intel VT-x** or **AMD-V** (sometimes labelled “SVM Mode” or just “Virtualization Technology”). On Windows also enable WSL 2 — Docker Desktop will offer to do this, and it needs a restart.

Ayan, on the login: **you do not need a Docker account for any of this**. Logging in is only required for pushing images or pulling private ones. Everything today pulls public images. Skip the login entirely and carry on.

We built everything in the IDE — so what is Docker actually doing here? And will we use it for the rest of the course?

MAHENDRA · ANONYMOUS

A good question, because today you genuinely saw both. When you ran `uvicorn` in your terminal, that was **your machine** running the code — no Docker involved. Docker did two things: it gave you a PostgreSQL server without installing one, and then, at `docker compose up --build`, it ran all three pieces together in containers so that one command starts the whole system.

The point of the second part is handover. Your laptop works because you spent three hours setting it up. A container works because the setup is written down and executed the same way every time. Yes — Compose is how we will run multi-service projects from here on, so today's file is a pattern you will reuse.

SECTION 4

Virtual environments, `requirements.txt` and `.gitignore`

What does “activating” an environment actually do internally?

EJAN

Less than almost everyone assumes, and this is worth knowing precisely. Activation modifies a couple of environment variables **in your current shell session only**. Mainly it prepends `.venv/bin` (or `.venv\Scripts` on Windows) to your `PATH`, so when you type `python`, the shell finds the one inside `.venv/` first. It also sets `VIRTUAL_ENV` and adjusts your prompt so you can see it.

That is genuinely it. No files are moved, nothing is installed, nothing outside that one terminal is touched. Two consequences follow directly: a second terminal is *not* activated, and closing the terminal or restarting your machine loses the activation but **not** the environment — the folder and all its packages are still on disk. Just run the `activate` command again.

Longer version, with the restart question answered specifically → **No stupid questions**, Part 2.

The activate command in the PDF is different from what you typed in the terminal

JISHAN BAIG

Both are in the guide — they differ by operating system, which is easy to miss when scanning:

```
source .venv/bin/activate      # macOS / Linux
.venv\Scripts\activate        # Windows
```

If Windows PowerShell refuses with “running scripts is disabled on this system”, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, accept, and try again.

Why is requirements.txt inside api/ and not at the project root? It is a requirement for the whole project.

MADHAV BHETANABHOTLA · JISHAN BAIG

You both asked twice, so I clearly did not answer it well live. Let me do it properly.

It is *not* a requirement for the whole project — it is the requirement list for **one service**. Look at the structure: `api/` is a Python service and its manifest is `requirements.txt`; `web/` is a Node service and its manifest is `package.json`. Each service declares its own dependencies, next to its own code.

There is also a hard technical reason. `api/Dockerfile` contains `COPY requirements.txt .`, and a Dockerfile can only copy from its **build context** — the folder given in `build: ./api`. A file at the project root is *outside* that context and cannot be copied. Move it up and the image build fails.

The general principle: one service, one folder, one dependency manifest, one Dockerfile. It keeps services independently buildable — which is exactly what lets you scale or deploy them separately later.

Can we rename requirements.txt to something else?

EJAN

Technically yes — `pip install -r anything.txt` works, the name has no magic. Practically, don't. Every Python developer, every CI system and every tutorial expects `requirements.txt`, and our Dockerfile names it explicitly. Renaming buys nothing and costs recognisability. (For what it's worth, modern Python is drifting toward `pyproject.toml` for this job — but that is a different conversation, not a rename.)

Is bin the equivalent of package.json?

SHUBH MANDALIA

No — easy confusion, but they are different kinds of thing. `.venv/bin/` holds *executables* (your python, pip, uvicorn, pytest); its closest Node equivalent is `node_modules/.bin/`. The equivalent of `package.json` — the declared list of dependencies — is `requirements.txt`.

venv creates a .gitignore by itself — why are we writing another one? And is .gitignore itself ignored when pushing?

AMAN ANAND · JISHAN BAIG

Aman, you spotted something real. Since Python 3.11, `python -m venv` writes a `.gitignore` *inside* `.venv/` containing `*` — a courtesy so the environment excludes itself. But it only covers that one folder. It knows nothing about `node_modules/`, `__pycache__`, `.next/` or `.DS_Store`, which is why we write one at the project root.

Jishan — no, `.gitignore` is itself committed and pushed, deliberately. It is a shared instruction: everyone who clones the repo gets the same exclusions. A `.gitignore` that only existed on your machine would be useless to your team.

Worth checking before your first push, because it catches this immediately:

```
git ls-files | head -30
```

You should see your own files and nothing from `.venv/` or `node_modules/`. If they are there, add them to `.gitignore`, then `git rm -r --cached .venv node_modules` and commit again.

SECTION 5

Ports, and the Docker commands that failed

How do we know whether a port is available?

KRISHNA SAGAR

Ask the machine who is holding it:

```
lsof -i :5432          # macOS / Linux
netstat -ano | findstr :5432    # Windows
docker ps              # is a container holding it?
```

A port is a numbered door on your machine, and only one program can listen at each. Ours: 3000 web, 8000 api, 5432 database. If something already has one, you do not have to fight it — remap. In `docker-compose.yml`, `"5433:5432"` means "port 5433 on my machine, 5432 inside the container". Change only the left number.

This matters especially if you already run PostgreSQL locally: your existing server holds 5432 and the container cannot publish onto it. Remapping the host port does **not** affect the api, because inside the Compose network it connects to `db:5432` — nothing to do with the published port.

Full walkthrough, including the silent version of this failure → build guide, **Part 4**, "If you already have PostgreSQL on your machine".

The long `docker run` command fails: "invalid reference format" / "includes invalid characters for a local volume name"

RISHI SHUKLA · ANONYMOUS

Both of you are on Windows, and both errors are the same root cause: **that command was written for a Mac/Linux shell**. Two pieces of it are not portable — the trailing `\` line continuations, and `$(pwd)`. Windows sees the backslashes as literal characters and never expands `$(pwd)`, so Docker receives mangled arguments. My omission for not giving you the Windows form clearly enough.

PowerShell, as one line, with `${PWD}`:

```
docker run --name intake-db -e POSTGRES_USER=intake -e POSTGRES_PASSWORD=intake
-e POSTGRES_DB=intake -v "${PWD}/db/init.sql:/docker-entrypoint-
initdb.d/init.sql" -p 5432:5432 -d postgres:16-alpine
```

Honestly though — **skip that command**. It exists only to show that you can run Postgres alone before writing any Compose file. If it is fighting you, go straight to `docker compose up --build`, which has none of these quoting problems because the paths live in the YAML file.

Where do I run the Docker commands — VS Code terminal or Docker Desktop's terminal?

ANONYMOUS

VS Code's integrated terminal, opened at your project folder. Docker Desktop is only there to keep the engine running and to give you a visual list of containers; you never need to type in it. Docker commands are relative to where you run them — `docker compose up` looks for `docker-compose.yml` in the current folder — so being in the project root matters.

Why create intake-db, then a db-1 appears, then you delete intake-db? And what was loan-intake1?

JISHAN BAIG

Reasonable confusion — that was me switching methods mid-flow without narrating it.

- `intake-db` was a **one-off container** I started by hand with `docker run`, purely to prove the API could talk to Postgres before any Compose file existed.
- `loan-intake-db-1` is the container **Compose** created. Compose names things `<project>-<service>-<number>` automatically.
- I deleted the hand-made one because both wanted port 5432, and two containers cannot hold the same port. Once Compose manages the database, the manual container is redundant: `docker rm -f intake-db`.

And `loan-intake1` was a second, empty folder where I cloned the repo fresh to prove it runs from a clean clone — that is Checkpoint 7 in the guide. It is not part of the project, which is why it is not in the PDF. Clone into any folder name you like.

SECTION 6

FastAPI and Pydantic

What does BaseModel contain? Is Pydantic defining the database model or the API?

ANONYMOUS · ANONYMOUS

Inheriting from `BaseModel` gives your class four things: parsing (JSON in, typed Python object out), validation against your annotations and `Field` constraints, serialisation back to JSON, and a JSON Schema used to generate the docs page.

And to the second question — **the API, not the database**. This is an important separation and today's project shows it cleanly. Pydantic models describe the shape of data crossing the network boundary. Your database schema is `db/init.sql`, a completely separate definition in SQL. They overlap in practice, but they are different contracts with different jobs: one protects your API, the other protects your data.

Is ApplicationIn a kind of middleware that applies to all endpoints?

ANUSHKA SEHRAWAT

No — and this is a useful distinction to draw. `ApplicationIn` is a **type on one parameter of one function**. It applies only where you write it. `create_application(payload: ApplicationIn)` means “this endpoint's body must be this shape”; the GET endpoints never mention it and are unaffected.

Middleware is the other thing — code that wraps *every* request, regardless of route. We used exactly one today: `CORSMiddleware`, added via `app.add_middleware(...)`. That is the mechanism you would use for authentication or request logging.

Where do we declare what is returned on success? What is the point of ApplicationIn versus ApplicationOut?

JISHAN BAIG

On the decorator — `response_model` is the success declaration:

```
@app.post("/applications", response_model=ApplicationOut, status_code=201)
```

That single line says: on success, return 201, and the body will match `ApplicationOut`. It also *filters* the response — anything not declared on the model is stripped before the client sees it, so internal fields cannot leak by accident.

The In/Out split is deliberate. `ApplicationIn` is what a client is allowed to **send**. `ApplicationOut` inherits those fields and adds `id` and `status` — the two things the **server** assigns. If you used one model for both, a client could submit its own `id` or set `status: "approved"`. Separating them makes that structurally impossible rather than something you have to remember to check.

Is the /docs page what people call Swagger?

VIKRANT SINGH

Yes. `/docs` is **Swagger UI**, rendering the OpenAPI description FastAPI generates from your type hints. Two things worth knowing: `/redoc` gives you the same content in a different, more readable layout, and `/openapi.json` is the raw machine-readable schema. That last one is the useful one — because it is a standard, other tools consume it to generate API clients, test suites or documentation sites automatically.

localhost:3000/docs gives an error, but localhost:3000 loads fine

PRATEEK SINGHAL

Correct behaviour — wrong port. `/docs` belongs to the **API**, so it is localhost:8000/docs. Port 3000 is the Next.js web page, which has no `/docs` route, so it correctly returns a 404. Two different programs on two different ports.

Is CORS where we block bots and unwanted IPs? Does the CORS check happen before or after authentication?

CHRISTY JOSE

A sharp question, and the answer is a firm no on the first half — this is a common and consequential misunderstanding, so worth being blunt: **CORS is not a security control against bots or attackers**. It is enforced by the *browser*, voluntarily. A bot, a script or `curl` simply ignores it. It exists to protect *your users* from a malicious page reading your API with their credentials — not to protect your API from bad clients.

For blocking bots and IPs you want different layers entirely: a WAF or CDN rule, rate limiting, and authentication.

On ordering: CORS middleware runs **before** your route and therefore before route-level authentication. Browsers also send a “preflight” `OPTIONS` request first for anything non-trivial, which must succeed *unauthenticated* — which is precisely why a misconfigured CORS setup looks like an auth failure and sends people down the wrong path for an hour.

SECTION 7

The database errors we hit live

“No module named db” / I don't have a .env file / how do I fix the DB error?

ANUSHKA SEHRAWAT · ANONYMOUS · RISHI SHUKLA

Taking these together, because they were the same pile-up.

“No module named db” — you are running `uvicorn` from the project root instead of from `api/`. `main.py` says `from db import get_conn`, which only resolves when `db.py` is in the working directory. Fix: `cd api` first, then `uvicorn main:app --reload`.

No .env file — correct, there isn't one, and you don't need one. I mentioned `.env` verbally as a general practice and that was confusing mid-exercise. `DATABASE_URL` comes from one of two places: the `environment:` block in `docker-compose.yml` when running under Compose, or the fallback default written into `db.py` when running locally. Nothing else to create.

Rishi — for the general DB error, work the troubleshooting table at the end of the build guide in order. Nine times out of ten it is one of: the container is not running (`docker ps`), `localhost` where it should be `db`, or `init.sql` never ran.

I get 201 Created, but GET returns [{}, {}, {}, {}] — empty objects

AKSHATA DV

This one is genuinely interesting, and the shape of it tells you the cause. Four empty objects means four rows *were* found — so your database and query are fine — but their fields did not map onto `ApplicationOut`, so `response_model` filtered every field out and left the envelopes.

Almost always one of two things in `db.py` / `main.py`:

- `row_factory=dict_row` is missing from `psycopg.connect(...)`. Without it rows come back as tuples, which have no field names to map.
- You are returning the raw rows instead of constructing models — it must be `return [ApplicationOut(**row) for row in rows]`.

Check both against the version-2 listing in Part 4 of the guide. It is a good error to have seen, because it teaches you what `response_model` is actually doing on the way out.

Pylance flags `dict_row` in `db.py`: “cannot be assigned to parameter `row_factory`”

SAYANTAN GHOSH

Your code is correct — **ignore this one**. That is Pylance in strict mode being pedantic about generics in `psycopg`'s type stubs; the pattern is straight from `psycopg`'s own documentation and runs fine. If the squiggle bothers you, add `# type: ignore[arg-type]` at the end of the `psycopg.connect(...)` line, or set `"python.analysis.typeCheckingMode": "basic"` in VS Code settings.

Worth naming the general lesson, since we spent the morning on type hints: a type-checker complaint is not a runtime error. Sometimes the stubs are simply imperfect. Learn to tell “my code is wrong” from “the annotations upstream are imprecise”.

How do I actually look at the data in my tables?

MOULI CHOUDHURY

Open a SQL prompt inside the running container:

```
docker compose exec db psql -U intake -d intake

\dt                -- list tables
SELECT * FROM applications;  -- see everything
\q                 -- quit
```

Note the semicolons — `psql` waits for one before running your SQL, which catches everybody once. If you prefer a GUI, point DBeaver, TablePlus or pgAdmin at `localhost:5432`, database `intake`, user `intake`, password `intake`.

How do we wire init.sql so it executes only once?

ANNUREKHA SB

You don't have to — the Postgres image does it. Anything mounted into `/docker-entrypoint-initdb.d/` runs automatically, but **only when the data directory is empty**, i.e. on a genuinely fresh database. That is why our compose file has:

```
volumes:
  - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
```

The important corollary: **it is not a migration system**. Editing `init.sql` later will not touch an existing database — a very common surprise. To re-apply it you must destroy the data and start clean: `docker compose down -v`, then `docker compose up`. Real projects use a migration tool (Alembic for SQLAlchemy) for schema changes over time.

SECTION 8

The web side

Why Next.js and not plain React?

SHOVIK DAS

Because React on its own is only the view layer — it renders components and nothing else. A real page also needs routing, a build setup, a dev server and a way to render on the server. React leaves all of that to you; Next.js includes it. Our whole web service is six files and one `npm run dev`, which with bare React would need a bundler config and a router wired up by hand.

It is also what Module 2 uses, so today's page is deliberately the same stack you will build on rather than a detour.

Can we use webpack with Next.js? And what is the difference between webpack config, next.config.js and tsconfig.json?

AJAY SARSWAT

Next.js *already* uses webpack internally (and increasingly Turbopack), so you are using it whether or not you think about it. You almost never configure it directly — and if you find yourself wanting to, that is usually a sign of fighting the framework. Three distinct jobs:

- `tsconfig.json` — how TypeScript type-checks and compiles: strictness, target, module resolution.
- `next.config.js` — how the *framework* behaves: redirects, image domains, environment, output mode. This is your normal knob.
- `webpack config` — how modules are bundled. Reached only through a `webpack: function` inside `next.config.js`, for genuinely unusual needs.

Can we use `src/` as the root instead of `app/`?

AJAY SARSWAT

Yes. Next.js supports `src/app/` with no configuration — move the folder and it is picked up. Many teams prefer it because it separates source from config files at the repo root. I used a bare `app/` to keep the tree short on screen. Either is idiomatic; be consistent within a project.

Why is there no build command in the web Dockerfile?

AJAY SARSWAT

A deliberate teaching shortcut, and you are right to notice it. Ours ends with `CMD ["npm", "run", "dev"]` — the development server, which recompiles on save. That is what you want while learning; it is the wrong thing to ship. Production would be:

```
RUN npm run build
CMD ["npm", "start"]
```

Better still, a multi-stage build: compile in one stage, copy only the built output into a slim runtime image. Smaller image, no build tools in production. That is on the stretch-goal list at the end of the guide.

SECTION 9

Shipping it to someone else

When someone else needs to run this, what exactly do I send them, and what command do they run?

MOULI CHOUDHURY · ANONYMOUS

You asked this three or four times and I kept half-answering it, so here it is properly. There are two different answers depending on what you are handing over.

Handing over source (what we did today). You send the **Git repository** — that is it. It contains `docker-compose.yml`, both Dockerfiles and all the source. They run:

```
git clone <your-repo-url>
cd loan-intake
docker compose up --build
```

You never send an image file. `--build` means Docker builds the images on *their* machine from the Dockerfiles, and pulls `postgres:16-alpine` from the public registry. This is why the README matters: that one command is the entire handover.

Handing over built images (production). You push images to a registry and send a compose file that references tags instead of build folders — covered in the next answer.

Don't we need to publish our product as an image? How do we update a version that has already shipped?

AJAY SARSWAT

Yes — for real deployment you build once and ship the artefact, rather than making every server rebuild from source. The cycle:

```
docker build -t myorg/loan-api:1.0.0 ./api
docker push myorg/loan-api:1.0.0
```

Their compose file then says `image: myorg/loan-api:1.0.0` instead of `build: ./api`, and they run `docker compose pull && docker compose up -d`.

To update, you do **not** modify a published image — images are immutable, and that is the point. You build a new one with a new tag (`1.0.1`), push it, and change the tag in the compose file. Which gives you the thing everyone actually wants: rollback is editing one line back to `1.0.0`. This is also why `:latest` is a trap in production — you lose the ability to say which version is running.

Can we scale the API and the database independently?

AJAY SARSWAT

The API, yes, and easily — because we kept it **stateless**. Every request opens its own connection and holds nothing in memory between requests, so any instance can serve any request:

```
docker compose up --scale api=3
```

(You would put a load balancer in front and drop the fixed host port to avoid collisions.)

The database, no — not that way. Databases hold state, so you cannot just run three.

Scaling one means read replicas, connection pooling, partitioning, or a managed service that handles it. This asymmetry is one of the most useful things to carry out of today:

stateless services scale by adding copies; stateful ones need a strategy. Keeping state out of your API is what buys you the easy option.

SECTION 10

Going deeper — good questions we ran past

Do we open a new database connection on every API call? Can that be optimised?

MAHENDRA · AJAY SARSWAT

Yes, we do — and you are right that it is the first thing to fix in anything real. Opening a connection costs a network round trip and a Postgres process; at load it becomes the bottleneck, and you will exhaust `max_connections`.

The answer is a **connection pool**: a set of connections opened once and lent out per request. With `psycopg 3`:

```
from psycopg_pool import ConnectionPool

pool = ConnectionPool(DATABASE_URL, min_size=1, max_size=10)

# in a route:
with pool.connection() as conn:
    rows = conn.execute("SELECT * FROM applications").fetchall()
```

You would create the pool once at startup (FastAPI's lifespan handler) and close it on shutdown. I left it out today because a pool adds lifecycle management to a session already carrying a lot — but it is the correct next step, and worth doing as a stretch goal.

When do I use `async def` versus `plain def`?

AJAY SARSWAT

A genuinely important question, and one people get backwards in a way that makes their API *slower*.

- **`def`** — FastAPI runs it in a thread pool, so blocking work does not stall the event loop. Correct for synchronous libraries.
- **`async def`** — runs on the event loop. Correct only when you `await` genuinely async I/O.

The trap: writing `async def` and then calling a *blocking* library inside it. That blocks the event loop for every other request simultaneously — worse than `plain def`.

Our endpoints are `plain def` and that is deliberate: `psycopg` as we use it is synchronous. To go async you would switch to `psycopg's` async connection (or `asyncpg`) and `await` the queries. Rule of thumb: **match the keyword to your library, not to fashion**.

Is there an ORM for Python we can use with FastAPI?

AJAY SARSWAT

Several. **SQLAlchemy** is the standard. **SQLModel** is built on it by FastAPI's own author and lets one class serve as both Pydantic model and table definition — a natural fit for what we built today. **Tortoise ORM** is async-first if you go that route.

We used raw SQL deliberately: with two queries, an ORM would have added a large abstraction while teaching you nothing about the actual conversation between service and database. Learn the conversation, then let the ORM shorten it.

Where do middleware like auth, logging and DB connections belong?

AJAY SARSWAT

Three different mechanisms, which is worth separating clearly:

- **Middleware** (`app.add_middleware`) — cross-cutting concerns on every request: CORS, request logging, timing headers.
- **Dependencies** (`Depends(...)`) — per-route needs, and where authentication really belongs. A `get_current_user` dependency runs before the route, raises 401 itself, and is testable in isolation.
- **Lifespan** events — startup and shutdown, where a connection pool is created and closed.

`Depends` is FastAPI's best feature and we did not get to it. It is the first thing to read about next.

What is the best FastAPI project structure? Should models, data access and routes live in one file?

AJAY SARSWAT · ANONYMOUS

Honest answer: for two endpoints, one file is correct, and splitting it would be worse.

Structure should follow size, not aspiration. But the direction it grows in is well established:

```
api/
├─ main.py          # app creation, middleware, router includes
├─ routers/         # endpoints grouped by resource
│   └─ applications.py
├─ schemas/         # Pydantic models (the API contract)
├─ models/          # database tables, if using an ORM
├─ services/        # business logic, no HTTP knowledge
├─ db.py            # connection / pool
└─ tests/
```

The rule that matters: **routers know about HTTP, services know about business rules, and neither should know about the other's concerns.** That separation is what makes the logic testable without spinning up a web server. Split when a file gets hard to navigate — not before.

When would I choose FastAPI over Django? Pros and cons?

AJAY SARSWAT

They solve different problems, and the choice is usually obvious once framed properly.

FastAPI — for APIs. Fast, async-native, typed, with automatic validation and docs, and small enough to hold in your head. It gives you no admin panel, no ORM, no auth system; you assemble those. Right for microservices, ML-serving endpoints, and anything a separate frontend consumes.

Django — for full applications. ORM, migrations, admin, auth and templates included and integrated. More opinionated, more to learn, less async-friendly historically. Right when you want a complete server-rendered product quickly.

For this course FastAPI is right precisely *because* it is small: nothing today was hidden by the framework.

Resources for learning FastAPI and building microservices?

AJAY SARSWAT

Start with the official tutorial at fastapi.tiangolo.com/tutorial — unusually good, and I would genuinely recommend it over any video course. Read the *Dependencies* and *Bigger Applications* sections in particular; they answer your structure and middleware questions in the framework's own idiom. Pydantic's docs at docs.pydantic.dev are worth an hour once the basics land.

On microservices — my honest advice is not yet. Get one service genuinely solid first: pooling, dependencies, tests, a real deployment. Most microservice pain comes from distributing a system before the single-service fundamentals are in place.

Does Docker have an AI agent?

AMIT MANCHANDA

It does, and it appeared in our session by accident — when Rishi's `docker run` failed, the output suggested `docker ai "help me fix this container error"`. That is **Ask Gordon**, built into recent Docker Desktop: it can explain errors, inspect your containers and suggest Dockerfile improvements. Useful for exactly the class of quoting error we hit today. Treat it as a helpful colleague rather than an authority — verify what it tells you.

SECTION 11

Course logistics — and what I owe you

We weren't working in parallel — will the guide alone get me through, including the database?

LAVANYA KOONALA

Yes, and this is exactly what that document is for. **Build the whole thing, end to end** is complete and standalone: prerequisites, folder setup, environment, all sixteen files in full with comments, the database, Compose, Git, and a troubleshooting table. Eight checkpoints tell you what you should see before moving on.

You do not need the recording. Work through it at your own pace, and if a checkpoint fails, note which one and what the error said — then message me. That is a far more answerable question than “it doesn't work”, and you will often solve it yourself while writing it down.

Please add the production nuances you mentioned to the PDF

JISHAN BAIG

Agreed, and noted — several came up verbally that deserve to be written down: connection pooling, Depends for auth, migrations instead of `init.sql`, multi-stage builds, named volumes, secrets handling, and pinned image tags. Most are now in this document's Section 10; I will fold them into the guide as a short “towards production” section. Thank you for asking — verbal asides are exactly what gets lost.

What packages will we need across the whole course, not just today?

THIRUKUMARAN K

Sensible request — installing once beats reinstalling weekly. I do not want to guess at a list and have you install the wrong things, so I am getting the consolidated package and tooling list from the programme team and will post it before Week 1 rather than answer from memory here.

What I can say confidently: Python 3.12 or 3.13, Docker Desktop, Git and VS Code cover everything for the next few sessions. Nothing you installed today is wasted.

You said “we'll cover this in a coming session”, but I don't see more sessions in my calendar

ANONYMOUS

That is a scheduling question rather than a technical one, and I do not want to give you a wrong date. I am raising it with the programme team and asking them to confirm the Week 1 schedule and invites to everyone directly. If your calendar still looks empty in the next day or two, please chase the programme team so it is tracked rather than assumed.

TensorFlow doesn't work on an Intel Mac — how do I handle that for future sessions?

NUPUR DICHWALKAR

Thank you for raising it twice; you were right to. It is outside today's scope, so I did not want to answer it badly on the fly.

Briefly: Apple's accelerated build (`tensorflow-macos` / `tensorflow-metal`) is Apple-Silicon-only, but standard TensorFlow does still run on Intel macOS on CPU — slower, and sometimes needing an older version. Realistically, for anything heavy on an Intel Mac, Google Colab is the pragmatic answer.

I am checking with the programme team whether later sessions need local TensorFlow at all, or whether Colab is the supported path. I will confirm before it becomes urgent — you should not be blocked by your hardware.

BEFORE WEEK 1

If your checkpoint did not finish today, that is genuinely fine — nobody is behind. Work through the build guide this week and get to **Checkpoint 7**: clone your own repo into a fresh folder and bring it up with one command. If that works, you are ready. If it doesn't, tell me which checkpoint failed and what the error said, and we will fix it before Week 1 starts.